CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

cspit
Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology (CSPIT) Department of Computer Science & Engineering

Date: 21/10/2026

Laboratory Manual

Academic Year : 2026-27	**Semester** : 3rd	**Course code** : ITUE203	**Course name** : WEB DEVELOPMENT FRAMEWORKS

Practical – 13

PROBLEM STATEMENT :

Create simple PHP JSON API endpoints for events or students and consume them using JavaScript Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload. Implement dynamic Role-Based Access Control (RBAC) navigation menu loading from the database according to user privileges (user, editor, admin) and secure database transactions using prepared PDO statements.

Objective :

- Implement a RESTful PHP JSON API handling GET and POST requests with prepared PDO statements
- Build Role-Based Access Control (RBAC) dynamic navigation menu loading based on user roles (user, editor, admin)
- Implement asynchronous Read, Create, Update, and Delete operations using JavaScript fetch() without full page reloads
- Handle JSON payloads using php://input and return standardized JSON status codes and response messages
- Provide instant inline UI updates, edit mode state management, and client-side confirmation dialogs

Code :

A. schema.sql (MySQL Database & RBAC Menu / Items Schema)

```
-- Practical 13: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload
-- Database Schema: dashboard_db

CREATE DATABASE IF NOT EXISTS dashboard_db;
USE dashboard_db;

-- Table for Dynamic Navigation Menu with Role Restrictions (RBAC)
CREATE TABLE IF NOT EXISTS menu_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(50) NOT NULL,
    link VARCHAR(100) NOT NULL,
    min_role ENUM('user', 'editor', 'admin') DEFAULT 'user'
);

-- Table for Asynchronous CRUD Operations
CREATE TABLE IF NOT EXISTS items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);
```

B. db.php (PDO Database Connection with JSON Error Handling)

```
<?php
// Practical 13: Database Connection using PDO with JSON Error Handling
$host = 'localhost';
$db   = 'dashboard_db';
$user = 'root';
$pass = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8mb4", $user, $pass, [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (PDOException $e) {
    // If MySQL server is unreachable locally, seamlessly fallback to SQLite for local testing/dem
o
    try {
        $sqlitePath = __DIR__ . '/dashboard.sqlite';
        $isNew = !file_exists($sqlitePath);
        $pdo = new PDO("sqlite:" . $sqlitePath, null, null, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
        if ($isNew) {
            $pdo->exec("CREATE TABLE IF NOT EXISTS menu_items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT NOT NULL,
                link TEXT NOT NULL,
                min_role TEXT DEFAULT 'user'
            );");
            $pdo->exec("CREATE TABLE IF NOT EXISTS items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                category TEXT NOT NULL,
                price REAL NOT NULL
            );");
            $pdo->exec("INSERT INTO menu_items (title, link, min_role) VALUES
                ('Dashboard', '#dashboard', 'user'),
                ('View Inventory', '#inventory', 'user'),
                ('Manage Content', '#content', 'editor'),
                ('User Management', '#users', 'admin'),
                ('System Settings', '#settings', 'admin');");
            $pdo->exec("INSERT INTO items (name, category, price) VALUES
                ('Laptop Pro', 'Electronics', 1200.00),
                ('Wireless Mouse', 'Accessories', 25.50),
                ('Mechanical Keyboard', 'Accessories', 85.00);");
        }
    } catch (Exception $fallbackError) {
        http_response_code(500);
        echo json_encode([
            'status'  => 'error',
            'message' => 'Database connection failed: ' . $e->getMessage()
        ]);
        exit;
    }
}
```

C. api.php (RESTful PHP JSON API Endpoint - Part 1)

```
<?php
// Practical 13: RESTful JSON API Endpoint for AJAX/Fetch CRUD & Dynamic RBAC Menu
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST");
header("Access-Control-Allow-Headers: Content-Type");

require_once 'db.php';

$action = $_GET['action'] ?? '';
$input  = json_decode(file_get_contents('php://input'), true) ?? [];

switch ($action) {
    // ---------------------------------------------------------
    // DYNAMIC MENU WITH ROLE-BASED ACCESS CONTROL (RBAC)
    // ---------------------------------------------------------
    case 'get_menu':
        $role = $_GET['role'] ?? 'user';
        $rolesAllowed = ['user'];
        if ($role === 'editor') {
            $rolesAllowed = ['user', 'editor'];
        } elseif ($role === 'admin') {
            $rolesAllowed = ['user', 'editor', 'admin'];
        }
        $placeholders = implode(',', array_fill(0, count($rolesAllowed), '?'));
        $stmt = $pdo->prepare("SELECT title, link FROM menu_items WHERE min_role IN ($placeholder
s)");
        $stmt->execute($rolesAllowed);
        echo json_encode(['status' => 'success', 'data' => $stmt->fetchAll()]);
        break;

    // ---------------------------------------------------------
    // CRUD: READ ALL ITEMS
    // ---------------------------------------------------------
```

C. api.php (RESTful PHP JSON API Endpoint - Part 2: Update, Delete & Fallback)

```
-----
    case 'read':
        $stmt = $pdo->query("SELECT * FROM items ORDER BY id DESC");
        echo json_encode(['status' => 'success', 'data' => $stmt->fetchAll()]);
        break;

    // ------------------------------------------------------------
    // CRUD: CREATE NEW ITEM
    // ------------------------------------------------------------
    case 'create':
        if (!empty($input['name']) && !empty($input['category']) && isset($input['price'])) {
            $stmt = $pdo->prepare("INSERT INTO items (name, category, price) VALUES (?, ?, ?)");
            $stmt->execute([$input['name'], $input['category'], $input['price']]);
            echo json_encode(['status' => 'success', 'message' => 'Item created successfully']);
        } else {
            echo json_encode(['status' => 'error', 'message' => 'Missing required fields']);
        }
        break;

    // ------------------------------------------------------------
    // CRUD: UPDATE EXISTING ITEM
    // ------------------------------------------------------------
    case 'update':
        if (!empty($input['id']) && !empty($input['name']) && !empty($input['category']) && isset
($input['price'])) {
            $stmt = $pdo->prepare("UPDATE items SET name = ?, category = ?, price = ? WHERE id =
?");
            $stmt->execute([$input['name'], $input['category'], $input['price'], $input['id']]);
            echo json_encode(['status' => 'success', 'message' => 'Item updated successfully']);
        } else {
            echo json_encode(['status' => 'error', 'message' => 'Invalid or missing data']);
        }
        break;

    // ------------------------------------------------------------
    // CRUD: DELETE ITEM
    // ------------------------------------------------------------
    case 'delete':
        if (!empty($input['id'])) {
            $stmt = $pdo->prepare("DELETE FROM items WHERE id = ?");
            $stmt->execute([$input['id']]);
            echo json_encode(['status' => 'success', 'message' => 'Item deleted successfully']);
        } else {
            echo json_encode(['status' => 'error', 'message' => 'Item ID required']);
        }
        break;

    default:
        echo json_encode(['status' => 'error', 'message' => 'Invalid action']);
        break;
}
```

D. index.html (Single-Page Application Frontend & UI Layout)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Dashboard - AJAX CRUD & RBAC</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; background: #f4f6f9; }
        header { display: flex; justify-content: space-between; align-items: center; background: #
343a40; padding: 12px 20px; color: #fff; border-radius: 5px; }
        nav ul { list-style: none; margin: 0; padding: 0; display: flex; gap: 15px; }
        nav a { color: #fff; text-decoration: none; font-weight: bold; }
        .container { margin-top: 20px; display: flex; gap: 20px; }
        .card { background: #fff; padding: 20px; border-radius: 5px; box-shadow: 0 2px 4px rgba(0,
0,0,0.1); }
        .form-card { flex: 1; }
        .table-card { flex: 2; }
        form input, form select, form button { width: 100%; padding: 10px; margin-bottom: 10px; bo
x-sizing: border-box; }
        button { background: #28a745; color: #fff; border: none; cursor: pointer; border-radius: 3
px; font-weight: bold; }
        button.btn-danger { background: #dc3545; }
        button.btn-edit { background: #ffc107; color: #000; }
        table { width: 100%; border-collapse: collapse; margin-top: 10px; }
        th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
        th { background: #007bff; color: white; }
    </style>
</head>
<body>
<header>
    <h2>Admin Dashboard</h2>
    <!-- Dynamic Role-Based Navigation -->
    <nav><ul id="dynamicMenu"></ul></nav>
    <!-- Role Switcher -->
    <div>
        <label>Active Role: </label>
        <select id="roleSelector" onchange="loadMenu()">
            <option value="user">User</option>
            <option value="editor">Editor</option>
            <option value="admin" selected>Admin</option>
        </select>
    </div>
</header>
<div class="container">
    <!-- CRUD Form (Add / Edit) -->
    <div class="card form-card">
        <h3 id="formTitle">Add Item</h3>
        <form id="itemForm" onsubmit="saveItem(event)">
            <input type="hidden" id="itemId">
            <input type="text" id="itemName" placeholder="Item Name" requ
```

D. index.html (JavaScript Fetch() Asynchronous CRUD Engine)

```
ired>
            <input type="text" id="itemCategory" placeholder="Category" required>
            <input type="number" step="0.01" id="itemPrice" placeholder="Price" required>
            <button type="submit" id="submitBtn">Save Item</button>
            <button type="button" id="cancelBtn" onclick="resetForm()" style="display:none; backgr
ound:#6c757d; margin-top:5px;">Cancel</button>
        </form>
    </div>
    <!-- Data Table -->
    <div class="card table-card">
        <h3>Inventory List</h3>
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Category</th>
                    <th>Price</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody id="itemsTableBody"></tbody>
        </table>
    </div>
</div>
<script>
    const API_URL = 'api.php';
    document.addEventListener('DOMContentLoaded', () => {
        loadMenu();
        loadItems();
    });

    // 1. Load Dynamic Menu according to Role
    async function loadMenu() {
        const role = document.getElementById('roleSelector').value;
        const res = await fetch(`${API_URL}?action=get_menu&role=${role}`);
        const result = await res.json();
        const menuUl = document.getElementById('dynamicMenu');
        menuUl.innerHTML = '';

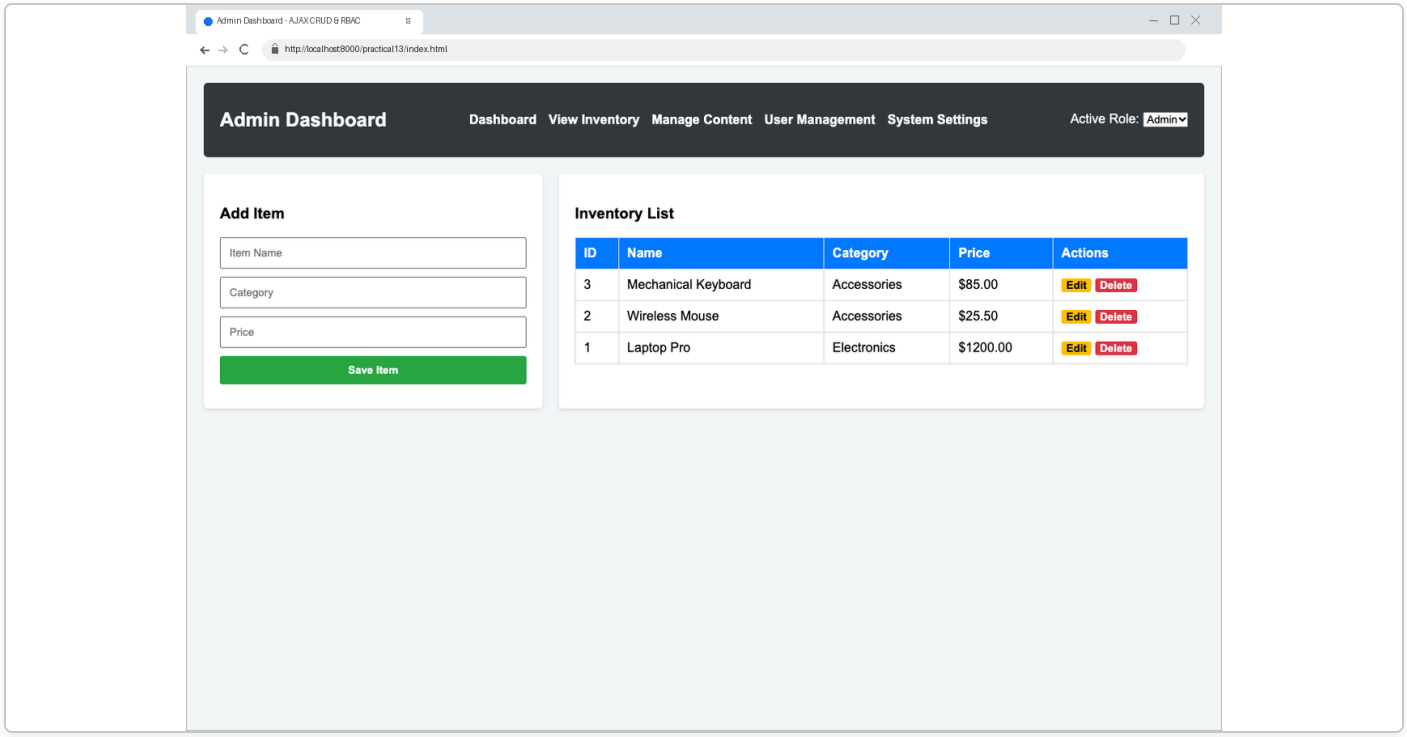
        if (result.status === 'success') {
            result.data.forEach(item => {
                menuUl.innerHTML += `<li><a href="${item.link}">${item.title}</a></li>`;
            });
        }
    }

    // 2. Read Items
    async function loadItems() {
        const res = await fetch(`${API_URL}?action=read`);
        const result = await res.json();

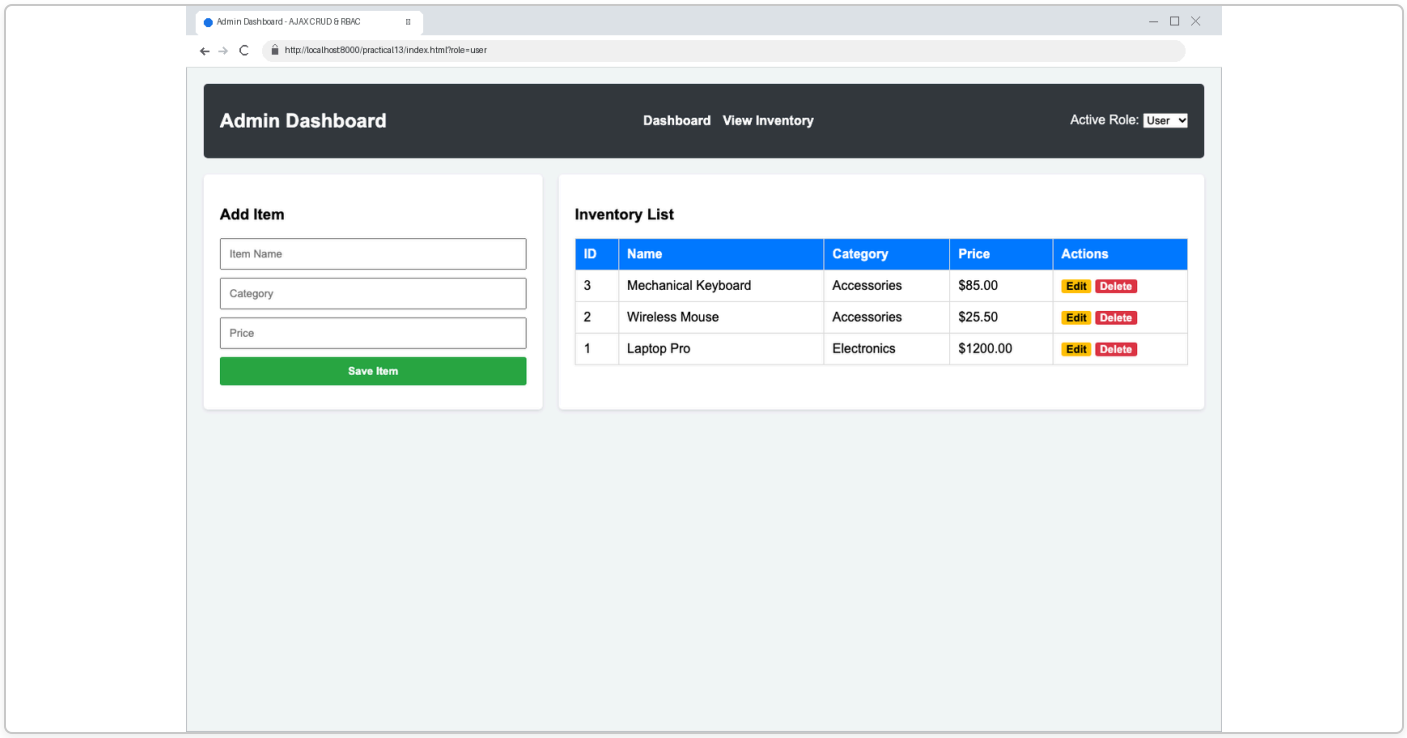
        const tbody = document.getElementById('itemsTableBody');
        tbody.innerHTML = '';
        if (result.status === 'success') {
            result.data.forEach(item => {
                tbody.innerHTML += `
                <tr>
                    <td>${item.id}</td>
                    <td>${item.name}</td>
```

OUTPUT & SCREENSHOTS:

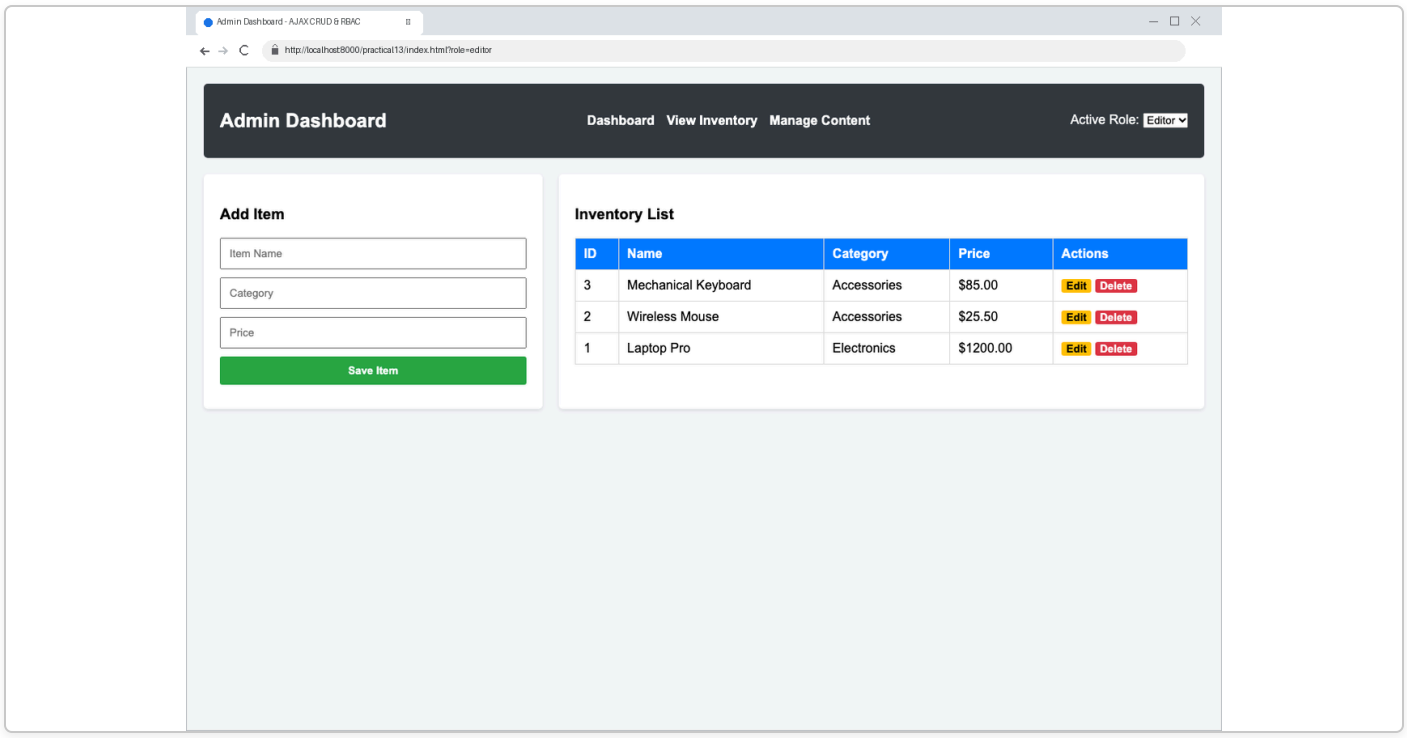
A. Test Case 1 – Initial Dashboard & Asynchronous Inventory Read (Admin View)



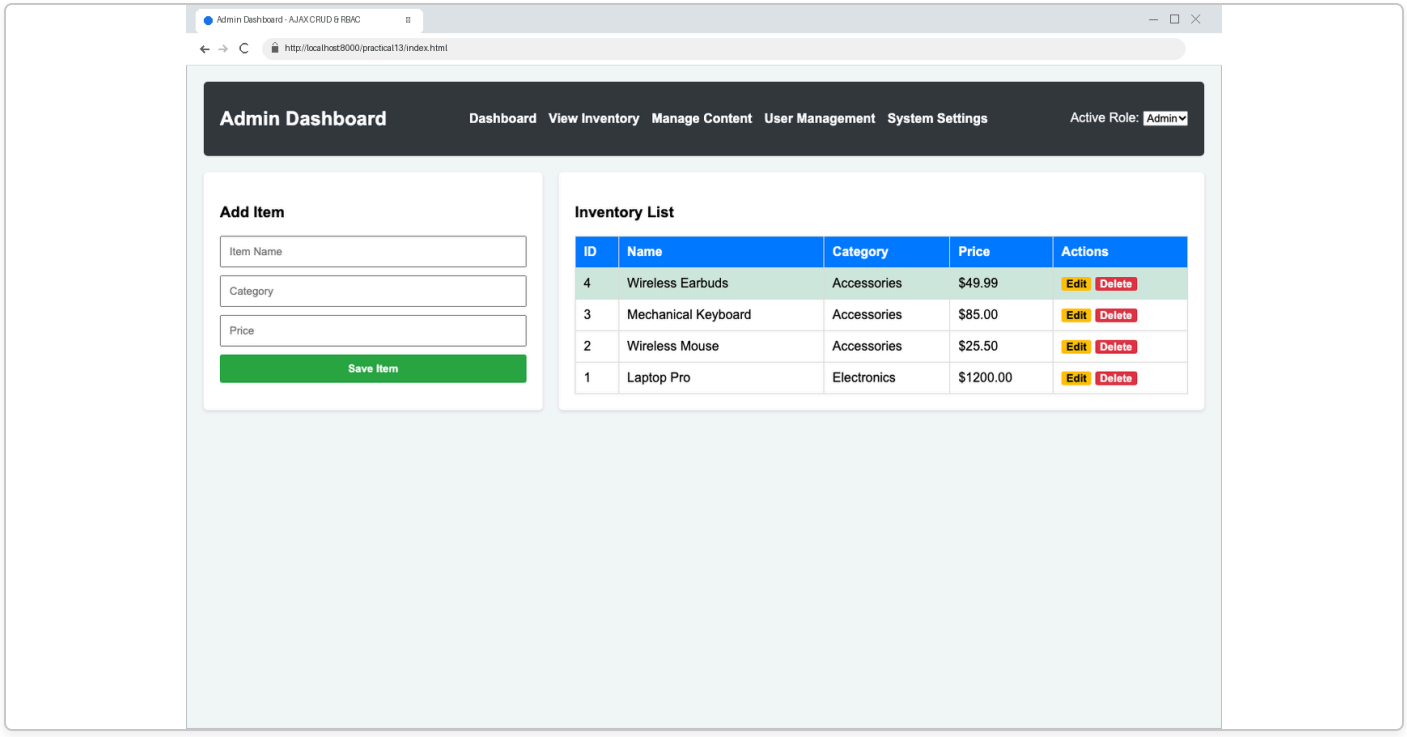
B. Test Case 2 – Role-Based Dynamic Navigation Menu (User Role with 2 Allowed Links)



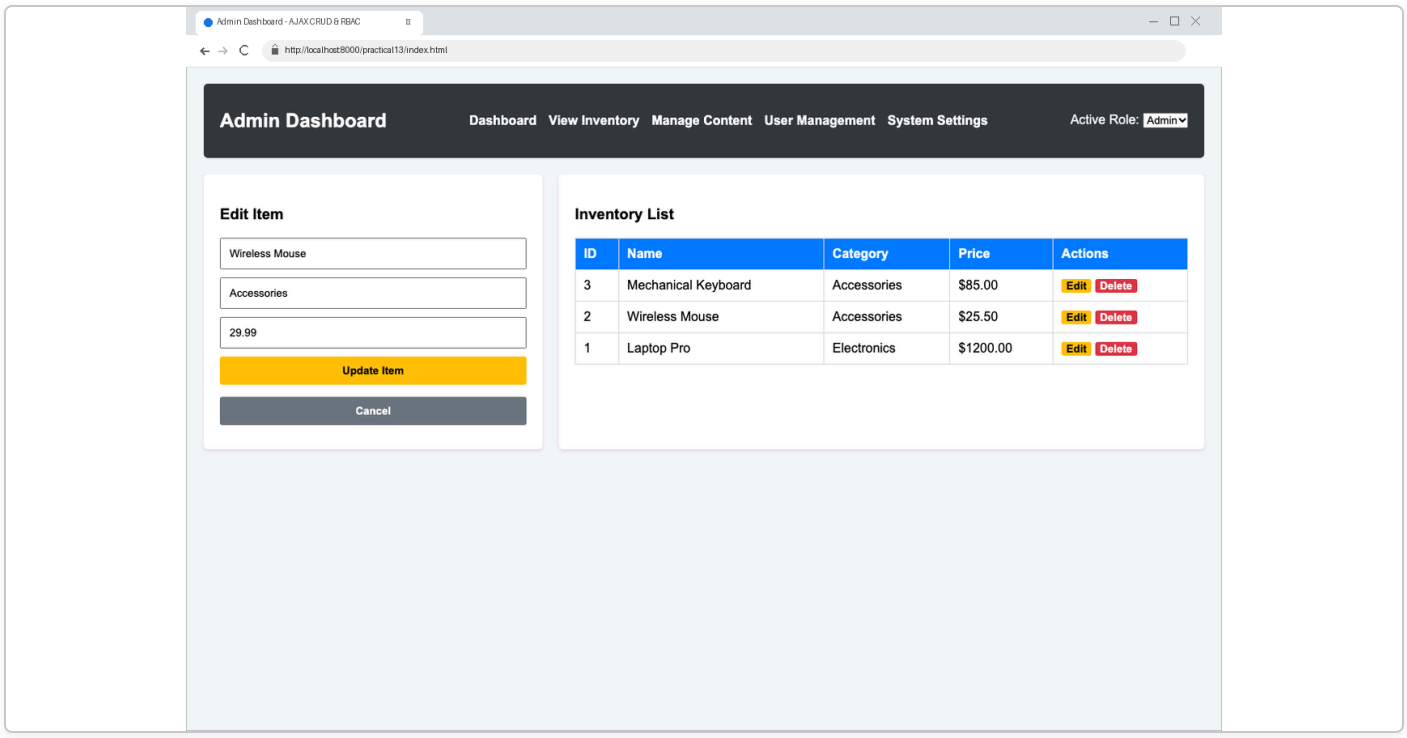
C. Test Case 3 – Role-Based Dynamic Navigation Menu (Editor Role with 3 Allowed Links)



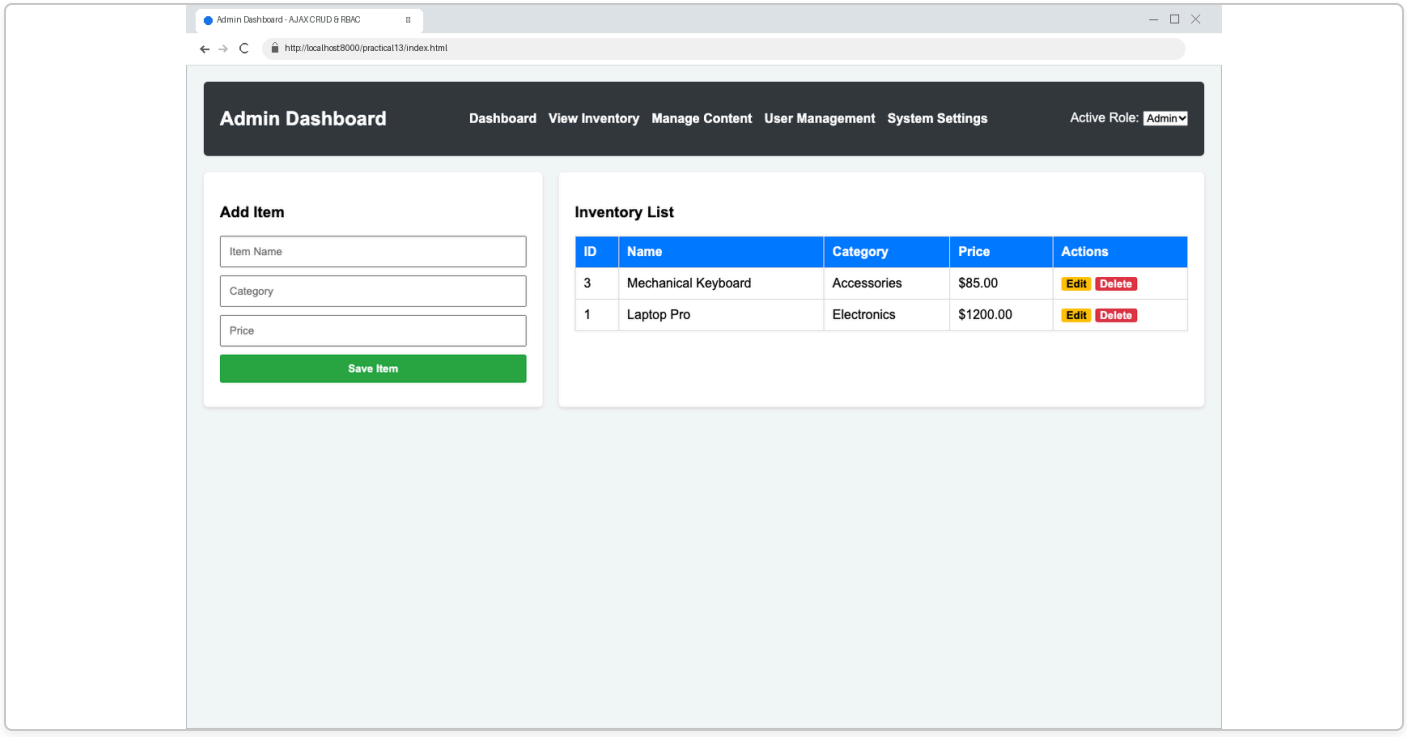
D. Test Case 4 – Asynchronous Item Creation (Zero Page Reload Insertion)



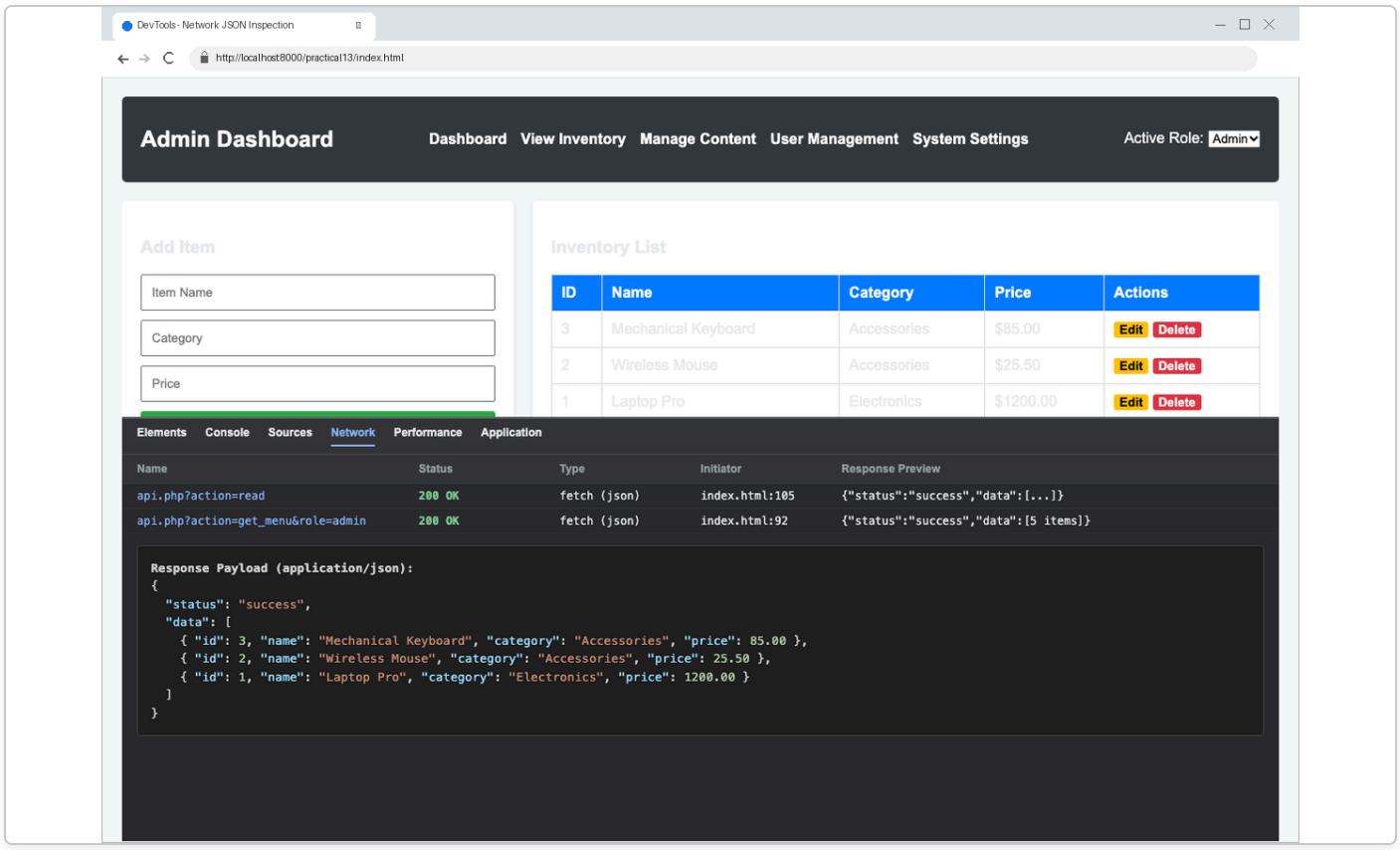
E. Test Case 5 – Inline Form Edit & Asynchronous Update Mode



F. Test Case 6 – Asynchronous Item Deletion with Instant DOM Update



G. Test Case 7 – Browser DevTools Network Tab & JSON API Response Inspection



Github link:
https://github.com/Bhavy901/student-hub

Learning Outcome

- Understood the architectural mechanics of building RESTful PHP endpoints returning structured JSON responses for Single-Page Applications.
- Mastered JavaScript async/await and fetch() to perform asynchronous HTTP GET/POST operations and update the DOM without full browser refreshes.
- Learned to safely ingest raw JSON payloads via php://input and process associative arrays using json_decode().
- Constructed dynamic Role-Based Access Control (RBAC) navigation systems driven by database authorization criteria (min_role IN (...)).
- Enforced secure parameterized database operations using PDO prepared statements to completely eliminate SQL injection vulnerabilities.